

HTML (Advanced)

1. Create a semantic blog page layout using <header>, <nav>, <article>, <aside>, and <footer> with dummy content.

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <title>Blog</title>

</head>

<body>

  <header>

    <h1>Blog</h1>

    <p>Welcome to Abhigna's Blog</p>

  </header>


  <nav>

    <a href="#">Home</a> |

    <a href="#">About</a> |

    <a href="#">Contact</a>

  </nav>


  <main>

    <article>

      <h2>Post</h2>

      <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec, pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim..</p>

    </article>
```

```
<aside>

  <h3>About</h3>

  <p>I am a blogger .Lorem ipsum dolor sit amet, consectetur adipiscing elit.
Aenean commodo ligula eget dolor. Aenean massa. Cum sociis natoque penatibus et
magnis dis parturient montes, nascetur ridiculus mus. Donec quam felis, ultricies nec,
pellentesque eu, pretium quis, sem. Nulla consequat massa quis enim.</p>

</aside>

</main>


<footer>

  <p>&copy; 2025 Blog. All rights reserved.</p>

</footer>

</body>

</html>
```

2. Implement an HTML5 video player with controls, autoplay, muted, and fallback text if the browser doesn't support <video>.

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8">

  <title>Video</title>

</head>

<body>

  <h1>video</h1>

  <video width="800" controls autoplay muted>

    <source src="video1.mp4" type="video/mp4">

    <source src="video2.webm" type="video/webm">
```

the browser does not support the video tag.

```
</video>
```

```
</body>
```

```
</html>
```

3. Build an HTML form that includes validation rules (required fields, email format, number range between 1–100).

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <title>Form</title>
```

```
</head>
```

```
<body>
```

```
<h1>HTML Form</h1>
```

```
<form>
```

```
  <label>
```

```
    Name: <input type="text" name="name" required>
```

```
  </label><br><br>
```

```
  <label>
```

```
    Email: <input type="email" name="email" required>
```

```
  </label><br><br>
```

```
  <label>
```

```
    Number: <input type="number" name="age" min="1" max="100" required>
```

```
  </label><br><br>
```

```
<button type="submit">Submit</button>

</form>

</body>

</html>
```

CSS (Advanced)

4. Using CSS Grid, create a 3x3 grid layout where the middle cell spans across 2 columns.

```
<!DOCTYPE html>

<html>

<head>

<style>

  body {

    margin: 0;

    font-family: calibri

    display: flex;

    justify-content: center;

    padding: 50px;

    background:lightblue

  }

  .grid {

    display: grid;

    grid-template-columns: repeat(3, 110px);

    gap: 5px;

    padding: 20px 0;

  }

  .grid div {
```

```

background: lightpink;

text-align: center;

padding: 20px 0;

justify-content: center;

font-weight: bold;
}

.middle {

    grid-column: span 2;

    background: lightblue;

}

</style>

</head>

<body>

    <h1>CSS Grid</h1>

    <div class="grid">

        <div>1</div> <div>2</div>

        <div>3</div> <div>4</div>

        <div class="middle">5</div>

        <div class="middle">6</div>

        <div>7</div> <div>8</div>

        <div class="middle">9</div>

    </div>

</body>

</html>

```

5. Design a responsive navigation bar using only CSS (without JavaScript) that turns into a hamburger menu on small screens.

```
<!DOCTYPE html>
```

```
<html lang="en">

<head>

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>CSS Navbar</title>

<style>

body

{ margin:0;

font-family: Calibri,

sans-serif; }

nav

{

background: #333;

color: white; }

nav ul

{

list-style: none;

margin: 0;

padding: 0;

display: flex;

}

nav li

{

padding: 10px; }

nav a

{

color: white;
```

```

    text-decoration: none; }

.menu-toggle
{ display: none; }

@media (max-width: 600px)
{
    nav ul
    {
        display: none;
        flex-direction: column;

    }

    .menu-toggle
    { display: block;
      padding: 18px;
      background: brown;
      cursor: pointer; }

    input[type=checkbox]:checked ~ ul { display: flex; }
}

</style>
</head>
<body>
<nav>

    <label class="menu-toggle" for="menu">&#9776; Menu</label>

    <input type="checkbox" id="menu" style="display:none;">

    <ul>

        <li><a href="#">Home</a></li>

        <li><a href="#">About</a></li>

```

```

    <li><a href="#">Services</a></li>
    <li><a href="#">Contact</a></li>
  </ul>
</nav>
</body>
</html>

```

6. Create a CSS animation where a circle moves left to right infinitely across the screen.

```

<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>A circle Moving from left to right</title>
<style>
  body {
    margin: 0;
    overflow-x: hidden;
    background: black }
  .circle {
    width: 100px; height: 100px;
    background: radial-gradient(circle,white);
    border-radius: 50%;
    position: relative;
    animation: slide 4s linear infinite;
  }
  @keyframes slide {
    0% { left: -60px; }

```



```

    100% { left: 100%; transform: translateX(-100%); }
  }
</style>
</head>
<body>
  <div class="circle"></div>
</body>
</html>

```

JavaScript (Advanced)

7. Write a JavaScript program to implement debouncing for a search input field (only trigger API call when user stops typing for 500ms).

```

<!DOCTYPE html>
<html>
<body>
  <h3>Debounce Search</h3>
  <input type="text" id="search" placeholder="Type input text here">
  <script>
    let time;
    document.getElementById("search").oninput = function(){
      clearTimeout(time);
      time = setTimeout(()=>{
        alert("Search: " + this.value);
      },500);
    };
  </script>
</body>

```

</html>

8. Implement a drag and drop feature where a user can drag an image into a drop zone.

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
  <h3>Drag and Drop an Image</h3>
```

```
  <div id="box" style="width:400px;height:400px;border:2px dashed #000;">
```

```
    Drop your image here
```

```
  </div>
```

```
<script>
```

```
let box=document.getElementById("box");
```

```
box.ondragover=e=>e.preventDefault();
```

```
box.ondrop=e=>{
```

```
  e.preventDefault();
```

```
  let file=e.dataTransfer.files[0];
```

```
  if(file && file.type.startsWith("image/")){
```

```
    let img=document.createElement("img");
```

```
    img.src=URL.createObjectURL(file);
```

```
    img.style.width="100%";
```

```
    box.innerHTML="";
```

```
    box.appendChild(img);
```

```
  }
```

```
};
```

```
</script>
```

```
</body>
```

```
</html>
```

9. Write a JavaScript function that fetches data from a public API (e.g., JSONPlaceholder) and dynamically displays it inside a list.

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8">

<title>Fetch data from API </title>

</head>

<body>

  <h2>Comments</h2>

  <ul id="dataList"></ul>


<script>

fetch("https://jsonplaceholder.typicode.com/comments?_limit=3")

  .then(response => response.json())

  .then(comments => {

    const ul = document.getElementById("dataList");

    comments.forEach(comment => {

      const li = document.createElement("li");

      li.textContent = `${comment.name} (${comment.email}): ${comment.body}`;

      ul.appendChild(li);

    });

  })

  .catch(err => console.error('Error fetching comments:', err));

</script>

</body>
```

```
</html>
```

10. Create a to-do list app in JavaScript where users can add tasks, mark them as completed, and delete them.

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<title>To-Do List</title>
```

```
</head>
```

```
<body>
```

```
<h2>To-Do List</h2>
```

```
<input id="task" placeholder="New task">
```

```
<button onclick="addTask()">Add</button>
```

```
<ul id="todo"></ul>
```

```
<script>
```

```
function addTask() {
```

```
  const input = document.getElementById("task");
```

```
  const value = input.value.trim();
```

```
  if (!value) return;
```

```
  const li = document.createElement("li");
```

```
  li.textContent = value;
```

```
  const compBtn = document.createElement("button");
```

```
  compBtn.textContent = "Completed";
```

```
  compBtn.onclick = () => li.style.textDecoration = "line-through";
```

```
const delBtn = document.createElement("button");

delBtn.textContent = "Delete";

delBtn.onclick = () => li.remove();

li.appendChild(compBtn);

li.appendChild(delBtn);

document.getElementById("todo").appendChild(li);

input.value = "";

}

</script>

</body>

</html>
```